

Clase 3: Funciones + condicional (if statement) + recursividad

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas M Victoria Tolosa

FUNCTIONS



Función

Una función es una **secuencia de instrucciones** que realiza un cómputo a la que se le ha asignado un nombre. Luego, este cómputo puede ser invocado llamando a la función por su nombre.

Para pensar

Soy un norteamericano que vive en Argentina y quiero pasar Celsius a Fahrenheit día a día.

$(20 * 9/5) + 32$

$(22 * 9/5) + 32$

$(18 * 9/5) + 32$

Realizar estos cálculos de manera directa en Python

Sintaxis de una función

Claramente podría hacer una función que haga el pasaje de Celsius a Fahrenheit.

```
def pasaje_c_f(t):  
    fahrenheit=(t*9/5)+32  
    return fahrenheit
```

- **def:** Palabra reservada por python para definir una función.
 - **pasaje_c_f:** Nombre de la función. Uno le puede poner cualquier nombre.
 - **t:** **input** de la función, que también puede tener cualquier nombre, lo que importa es lo que significa.
 - **return:** palabra clave para explicitar el **output** de la función.
- Hay 2 elementos ocultos que estamos ignorando. **Se dan cuenta cuáles son?**

Sintaxis de una función

- **“:” + indentación:** La manera que tiene Python de decir “estas líneas de código pertenecen a esta función que se define más arriba” es mediante indentación. La misma es una suerte de “sangría”. También la utilizaremos al hablar del **for** y **while** loop más adelante

Para pensar

- Es necesario que una función tenga uno o más inputs?
- Es necesario que una función tenga uno o más outputs?

Ejercicio

Crear una función que printee por pantalla tu nombre y apellido

```
def presentacion():  
    print('Soy Pablo J Piccoli Bornia, mi DNI es: 3x.xxx.xxx')  
  
def presentacion_2():  
    return 'Soy Pablo J Piccoli Bornia, mi DNI es: 3x.xxx.xxx'
```

**Para pensar: Cuál es el tipo de datos de m?
Cuánto vale? Qué vemos por pantalla?**

situación1:

```
presentacion()  
m=presentacion()  
type(m)
```

situación1:

```
presentacion_2()  
m=presentacion_2()  
type(m)
```

Ejercicio: Reloj californiano

Codear una function cuyo input sean las horas y minutos en Buenos Aires y su output sea su hora y minutos en California

Por qué usar functions?

- Expandir el lenguaje con nuevas funcionalidades/herramientas.
- Construir programas en base a piezas simples, pequeñas.
- Ordenar conceptualmente el código.
- Reutilización del código: construir nuevos programas en base a otros existentes.
- Facilitar el debug: búsqueda (y corrección) de errores en un único lugar.
- Facilitar el mantenimiento del código a largo plazo.

Variables locales

```
def dolar_a_peso(dolar):  
    peso=1450*dolar  
    return peso  
  
sueldo_total=dolar_a_peso(500)+500000  
extra=sueldo_total-peso
```

- Qué hace la function?
- Qué estoy tratando de calcular?
- Por qué esto falla?

Variables locales

- Toda variable definida dentro de una función tiene validez sólo dentro de esa función. Se la llama variable local.
- La manera de “sacar hacia afuera” una variable local es retornarla básicamente

Orden de parámetros y contexto

Una función no tiene manera de sacar por contexto el significado de un input. Para que todo funcione como se pretende, la manera es pasar los inputs en orden

```
#variables
nota='10'
nombre='Juan Perez'

#Function
def certificado(alumno,calificacion):
    print('Certifico que el alumno '+alumno+' asistió al parcial y obtuvo una calificación de '+calificacion+'/10')

certificado(nota,nombre)
```

Este script arroja un error? Retornea lo que queremos?

Función main

Hay varias maneras de ejecutar un script. La cátedra sigue la siguiente estructura.

```
#Importar librerías
import math as mt

#Definir funciones

def circunf(r):
    return 2*mt.pi*r

#Una función main donde se definen las variables
#Y la secuencia de ejecución
def main():
    radio=2
    circunferencia=circunf(radio)
    print('La circunferencia es ',str(circunferencia))

#Inicializador
if __name__ == "__main__":
    main()
```

Función main

- Al comenzar la ejecución, python realiza un screen de todas las functions creadas.
- Acto seguido, nota el inicializador y triggerea la function main()
- La idea es que main sea la function donde la lógica del script sea explicada y las variables sean definidas.

Reutilización

- Si prestaron atención, **invocamos una función dentro de otra.**
- Esto es perfectamente factible, se llama reutilizar

```
def cambio_divisa(dolar, tipo_cambio):  
    peso=dolar*tipo_cambio  
    return peso  
  
def dinero_ganado(dolar_hoy, dolar_ayer, tipo_cambio_hoy, tipo_cambio_ayer):  
    dinero_hoy=cambio_divisa(dolar_hoy, tipo_cambio_hoy)  
    dinero_ayer=cambio_divisa(dolar_ayer, tipo_cambio_ayer)  
    return dinero_hoy-dinero_ayer  
  
def main():  
    d_hoy=1900  
    d_ayer=1890  
    t_p_hoy=1450  
    t_p_ayer=1500  
  
    ganancia_neta_pesos=dinero_ganado(d_hoy, d_ayer, t_p_hoy, t_p_ayer)  
  
if __name__ == "__main__":  
    main()
```

Qué hace esta pieza de código?

Ejercicio

Realizar un script que calcule el retorno de comprar una acción y luego venderla. Inputs: valor al comprar y al vender.

ESTRUCTURA CONDICIONAL (IF STATEMENT)

Condicional (if)

La cláusula if te permite alterar el flujo de ejecución dependiendo del resultado de una operación booleana.

```
import random

var=random.random()

if var > 0.8:
    print('Tocó un valor alto')
elif var < 0.2:
    print('Tocó un valor bajo')
else:
    print('Dentro del promedio')
```

Qué dicen que hace esta pieza de código?

Condicional (if)

- If es una palabra reservada que sirve para plantear una condición.
- En caso de que la operación booleana que esta inmediatamente después (corolario) sea verdadera, se ejecuta lo que está indentado dentro.
- **Puede** haber otras palabras que formen parte de la misma estructura condicional, todas ellas con su propia indentación: **elif y else**.
- Elif, al igual que el if, tiene un corolario. Else no, ya que else es la opción de descarte

Ejercicio sencillo

Crear una function que reciba 2 números y devuelva el mínimo entre los 2.

Sobre el if

```
var=50

if var>10:
    print('Es mayor a 10')
elif var>20:
    print('Es mayor a 20')
```

```
var=50

if var>10:
    print('Es mayor a 10')
if var>20:
    print('Es mayor a 20')
```

Qué printea cada bloque de código por pantalla?

Sobre el if

- Cada vez que tenemos un if, comienza una estructura condicional nueva, 2 ifs no es equivalente a un if+ un elif

Sobre el if

Y aca? Hay alguna diferencia en correr estas 2 piezas de código?

```
var=50
if var>=0:
    if var<100:
        print('Es un número natural menor a 100')
```

```
var=50
if var>=0 and var<100:
    print('Es un número natural menor a 100')
```

- En la primera estamos viendo un if **anidado**.
- En la 2da estamos viendo un if con un **corolario de 2ble condición**.

Sobre el if y el return

Qué piensan que sucede en esta línea de código?

```
def decisor(val):  
    if val>50:  
        return val*2  
    else:  
        val=val+50  
        return val/2  
  
decisor(51)
```

Cuando la ejecución de una función se topa con un return, devuelve el valor interrumpiendo la operatoria

DIVISIÓN ENTERA Y RESTO

2 operadores importantes

- La **división entera** `"//"` devuelve la parte entera del float que se genera al realizar una división. Siempre redondea hacia abajo.
- El **resto** `"%"` devuelve, valga la redundancia, el resto de una división entera entre 2 números.

Ejercicio

- Crear un script que devuelva horas y minutos a partir de minutos.
- Ej: Input 105 minutos. Output 1 hs y 45 min

MÓDULOS, LIBRERÍAS Y PAQUETES

Built - in functions

- Es importante notar que, si bien recién hoy nos enteramos de lo que una function es y como crearla, ya venimos usando funciones.
- `type()`, `print()`, `str()`
- Estas son llamadas built-in functions y son funciones pre-programadas que ya vienen dentro de python

Módulo/librería/paquete

- Usaremos los 3 términos como sinónimos.
- Son un conjunto de variables, constantes y funciones empaquetados en una carpeta. El mismo es mantenido por una comunidad y puede ser importado a tu código en python.
- Hay **muchas maneras** de llamar a una librería para usarla en tu código.

Import

```
#Se importa todo el módulo con su nombre original  
import math
```

```
#Printea el número Pi  
math.pi
```

```
#Se importa todo el módulo con un alias  
import math as mt  
  
#Printea número pi  
mt.pi
```

```
#Se importa solo lo que se necesita  
from math import pi  
  
pi
```

```
from math import *  
  
pi  
  
e #Número de Euler, también lo importaste
```

Qué potencial problema
tiene la importación sin
alias ni nombre completo?

Crear tu propio módulo

```
def fun_1(x):  
    return x**x  
  
def fun_2(x):  
    return 3*x+5
```

Guardo esto en mi
en un script
llamado
funciones_locas.p

```
import funciones_locas as fl  
  
##Esto da 256  
fl.fun_1(4)
```

✓ En la misma carpeta puedo crear
otro script e importar estas
funciones dentro de un paquete
que se llamará igual que el .py
donde guardaste las funciones.

Ejercicio dificultad alta

Crear un script que nos diga si este año en el que estamos es o no bisiesto.

RECURSIÓN



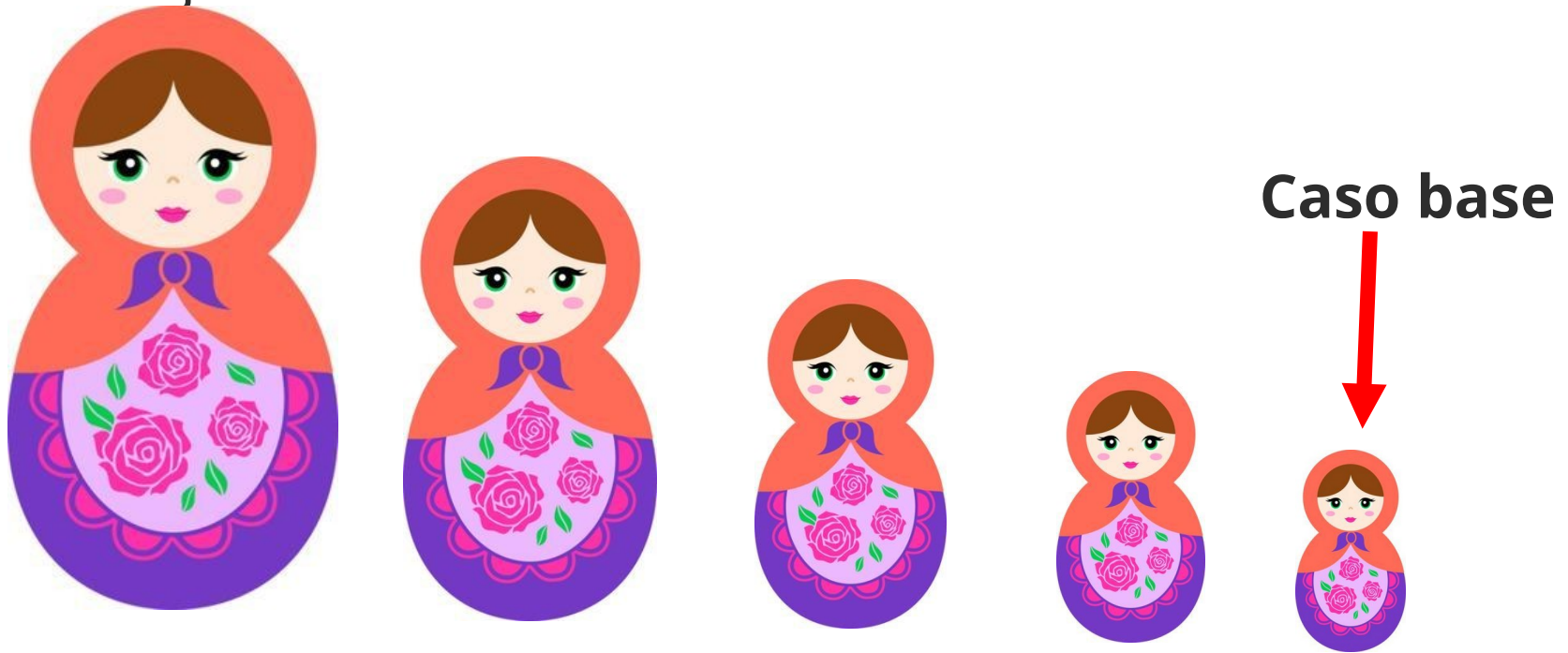
Recursión

Como vimos en la intro a la materia, unas de las cosas que siempre se busca hacer computacionalmente son cálculos iterativos que costaría mucho realizar uno a uno.

- Una forma de hacerlo es creando estructuras que llamamos loops (los veremos en la próxima clase)
- Otra forma de hacerlo es mediante una técnica llamada recursión.

Recursión

Una función recursiva es una función que se llama a sí misma hasta tocar un caso base. Este caso base delimita el fin de la ejecución.



Ejemplo: calcular el factorial de un número

$$n! = \prod_{i=1}^n i = 1 \times 2 \times 3 \times \cdots \times n.$$

Osea:

$$9! = 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$$

$$3! = 3 \times 2 \times 1$$

$$12! =$$

$$12 \times 11 \times 10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$$

- Cuánto es $0!$?
- Hay factorial de números no-naturales?

Ejemplo: calcular el factorial de un número

```
def fact(n):  
    if n==0:  
        return 1  
    else:  
        return n*fact(n-1)
```

Qué podemos ver aca?

- Como ya dijimos, la function se llama a sí misma
- La function devuelve un output (return) a cada iteración.
- El primer if delimita el caso base

Ejemplo: calcular el factorial de un número

```
def fact(n):  
    if n==0:  
        return 1  
    else:  
        return n*fact(n-1)
```

f(6). Procedimiento, estos pasos va realizando el algoritmo:

- $6 * \text{fact}(5)$
- $6*5*\text{fact}(4)$
- $6*5*4*\text{fact}(3)$
- $6*5*4*3*\text{fact}(2)$
- $6*5*4*3*2*\text{fact}(1)$
- $6*5*4*3*2*1*\text{fact}(0)$
- $6*5*4*3*2*1*1 = 720$

Qué pasa si tenemos una función recursiva y no tenemos ningún caso base?

Ejercicio dificultad alta

Crear una function que devuelva el n puesto de la serie de Fibonacci

Ej:

$\text{fibo}(8) = 21$

$\text{fibo}(7) = 13$